

Document Number: P0725R0
Date: 2017-07-03
Audience: Evolution Working Group
Reply-to: Tom Honermann <tom@honermann.net>

Remove the requirement for constrained-type-specifiers to be deduced to the same type from the Concepts TS

- [Introduction](#)
- [Terminology](#)
- [Motivation](#)
- [Wording](#)
- [Acknowledgements](#)
- [References](#)

Introduction

The requirement that each occurrence of a *constrained-type-specifier* appearing in the type of a variable declaration, return type of a function declaration, type of a parameter in an abbreviated function declaration or generic lambda expression, or the type of a deduction constraint be deduced to the same type has been controversial as evidenced by various discussions within the committee reflectors, positions stated by committee members in advance of the Toronto 2017 meeting, and the [P0464R2](#) [\[P0464R2\]](#) and [P0691R0](#) [\[P0691R0\]](#) paper submissions. This paper proposes removing the same type deduction requirement from the Concepts TS with the goal of increasing consensus on adopting the remaining Concepts TS functionality into the current working paper.

Removal of support for the same type deduction requirement means that the following examples that are currently ill-formed according to the Concepts TS will now be well-formed.

```
template<typename> concept bool C = true;
template<typename, typename> class ct {};

ct<C,C> v1 = ct<int,char>{};           // Previously ill-formed; each occurrence of C is
                                       // not deduced to the same type. Now ok.

ct<C,C> f1() {
    return ct<int,char>{};             // Previously ill-formed; each occurrence of C is
                                       // not deduced to the same type. Now ok.
}

ct<C,C> af1(C) {
    return ct<int,char>{};
}

auto v2 = af1(0L);                     // Previously results in an instantiation error due
                                       // to C being deduced to long from the function
                                       // argument and ct<int,char> not being convertible
                                       // to ct<long,long>. Now ok; the return type is
                                       // deduced to ct<int,char>.

auto gl1 = []{ return C; }{};
auto v3 = gl1(0, 'c');                 // Previously results in an instantiation error due
                                       // to different types being deduced for each occurrence
                                       // of C in the generic lambda expression's parameter
                                       // list. Now ok.

template<typename T> concept bool X =
    requires (T t) {
        { t } -> ct<C,C>;
    }
```

```
};
static_assert(X<ct<int,char>>); // Previously fails; ct<int,char> doesn't satisfy
                                // the deduction constraint. Now ok.
```

This change would affect, perhaps silently, the behavior of existing code. Consider the following example:

```
C af2(int i, C) {                // Previously, the return type was deduced from the function
    return i;                    // argument. Now, it is deduced from the return statement.
}
auto v3 = af2(0, 0L);           // Previously resulted in v3 having type long.
                                // Now v3 is deduced to have type int.
```

Additionally, the following code would become ill-formed due to the return type no longer being deducible.

```
C af3(C) {                      // Previously ok; the return type matches the deduced type
    return {};                  // for the function argument. Now ill-formed since the
}                               // return type is not deducible from the return statement.
```

This paper does **not** propose changing the requirement that the same type be deduced when multiple declarators are present [*Note*: gcc 7.1.0 and earlier fail to diagnose differently deduced types for each declarator when the declared type includes *constrained-type-specifiers*. See [gcc bug 81270](#) [[GccBug81270](#)] — end note]:

```
ct<C,C> v4 = ct<int,char>{}, // Previously ill-formed; different types are deduced
v5 = ct<int,long>{};        // for the types of v4 and v5. Remains ill-formed.
```

Terminology

Some of these terms have been adopted from [P0694R0](#) [[P0694R0](#)].

- **Consistent resolution**

Consistent resolution describes the current behavior of the Concepts TS in which repeated use of a *constrained-type-specifier* within a declaration requires that the same type be deduced for each occurrence. That is, given `ct<C,C>` from the introduction, a call to the following invented function template must succeed.

```
template<C T> void ft(ct<T,T>);
```

- **Independent resolution**

Independent resolution describes the alternate behavior proposed in this paper in which repeated use of a *constrained-type-specifier* within a declaration allows each occurrence to be deduced to the same or a different type. That is, given `ct<C,C>` from the introduction, a call to the following invented function template must succeed.

```
template<C T1, C T2> void ft(ct<T1,T2>);
```

- **Matched resolution**

Matched resolution describes behavior not currently available (outside a function template declaration) in which a type or portion thereof can be specified as the type deduced for a *constrained-type-specifier* present in some deducible context within the declaration. The extensions proposed in [N3878](#) [[N3878](#)] provides such support. Given `ct<C{T},T>`, a call to the following invented function template must succeed.

```
template<typename T> using identity = T;
template<C T> void ft(ct<T, identity<T>>);
```

Motivation

[P0464R2](#) [\[P0464R2\]](#) provides motivation for removing the same-type deduction requirement (*consistent resolution*) for abbreviated functions in favor of allowing different-type deduction (*independent resolution*). That motivation is acknowledged and little further motivation with respect to abbreviated functions is provided here. The following discusses additional motivation relative to the types of variable declarations, return types of non-abbreviated functions, and deduction constraints.

The requirement for types specified by *constrained-type-specifiers* to be deduced to the same type within a declaration (*consistent resolution*) was introduced based on an expectation that such resolution would be frequently desired and that non-abbreviated declarations could be used when not desired. While certainly true that *consistent resolution* is often desired, it is also true that it is often not desired. Use of non-abbreviated declarations does allow precise specification, but only for function template declarations; there is currently only one syntax available for variable declarations, non-template function declarations, and deduction constraints.

Consider the following variable declaration copied from the introduction. The declaration is currently ill-formed because the deduced types for both occurrences of `C` do not match.

```
ct<C,C> v1 = ct<int,char>{}; // Currently ill-formed.
```

There are several solutions available for making this well-formed such that each of the arguments to `ct` are individually deduced, but still required to satisfy `C`. For example:

```
template<C T1, C T2> void ctCC_with_independent_resolution(ct<T1,T2>);
template<typename T> concept bool ctCC =
    requires (T t) {
        ctCC_with_independent_resolution(t);
    };

ctCC v1 = ct<int,char>{}; // Ok; ctCC allows independent resolution.
```

[P0694R0](#) [\[P0694R0\]](#) suggests introducing additional names for the same concept to enable *independent resolution*. For example:

```
template<typename T> concept bool C2 = C<T>; // A duplicated concept definition.

ct<C,C2> v1 = ct<int,char>{}; // Ok; independent resolution
```

Neither of these solutions is satisfactory. Of course, should the behavior be changed to *independent resolution* as proposed in this paper, then enabling *consistent resolution* to preserve the current behavior is equally unsatisfactory (though necessary only if it is desired to explicitly reject deduction of different types):

```
template<C T> void ctCC_with_consistent_resolution(ct<T,T>);
template<typename T> concept bool ctCC =
    requires (T t) {
        ctCC_with_consistent_resolution(t);
    };

ctCC v1 = ct<int,char>{}; // Ill-formed (as intended); ctCC requires consistent resolution.
```

The extensions proposed in [N3878](#) [\[N3878\]](#) provide additional flexibility, but do not provide the full range of expression needed. For example:

```
ct<C{T},T> v1 = ct<int,char>{};    // Not proposed; syntax from N3878.
                                   // Ill-formed (as intended); C is deduced to be int for the
                                   // first template argument, the second argument is declared
                                   // to match the deduced type for the first argument,
                                   // and ct<int,char> is not convertible to ct<int,int>.
```

For full genericity, it appears extensions like those proposed in both [N3878](#) ^[N3878] and [P0694R0](#) ^[P0694R0] would be needed to enable expressing:

- Consistent resolution:

```
ct<C{T},C{T}> v1 = ...; // Such that both occurrences of C require matching deduced types.
```

- Independent resolution:

```
ct<C,C> v1 = ...;      // Such that each occurrence of C is independently deduced.
```

- Matched resolution (enabling conversions):

```
ct<C{T},T> v1 = ...;   // Such that C is deduced from the first template argument,
                        // but not the second.
```

The above examples are illustrated with variable declarations, but the same concerns apply to *constrained-type-specifiers* in function return types and deduction constraints as well. Consider the following example constraint written to require an expression to have a type requiring `std::tuple` with a specific arity, but constrained element types. With the changes proposed in this paper, *independent resolution* will allow the element types to be deduced to the same or different types. If *consistent resolution* is desired, then that additional requirement can be added with the (admittedly unsatisfactory) techniques discussed earlier.

```
template<typename T> concept bool V =
    requires (T t) {
        { t } -> std::tuple<C,C,C>;
    };
```

Abbreviated functions have the behavior that *constrained-type-specifiers* specified within the return type of the function that match a *constrained-type-specifier* used in a parameter type are not deduced from non-discarded return statements from the body of the function (as is the case when the return type is `auto` or `decltype(auto)`), but are rather replaced by the deduced type for the matching parameter(s). This behavior introduces the possibility that a change to the function's parameter list may alter the meaning of the return type. For example, if this function:

```
C af4(auto p1);
```

is modified to add a parameter `p2` with the *constrained-type-specifier* `C`:

```
C af4(auto p1, C p2);
```

then the change to the parameter list will result in the return type matching the deduced type for `C` from the parameter list rather than being deduced from return statements in the function definition. Such an effect is subtle and may be made unintentionally. Extensions like those discussed above would enable making these relationships explicit when desired:

```
auto af4(auto p1, C{T} p2) -> T; // The return type matches the type deduced
                                // for parameter p2.
```

[P0694R0](#) [\[P0694R0\]](#) argues that *consistent resolution* should be the default for variables declared with equivalent *constrained-type-specifiers* within the same scope and provides the following example (where `Number` names a concept):

```
Number operator+(Number, Number);

template<typename T> concept bool Number2 = requires Number<T>;

void f()
{
    Number    x = 1+2;      // OK
    Number    y = 1.0+2.0; // error: different type bound to same concept in a single scope
    Number2   y = 1.0+2.0; // OK
}
```

However, *consistent resolution* as portrayed in this example seems unworkable as a default for variable declarations at namespace scope since the first variable declaration would lock in the deduced type associated with the *concept-type-specifier* for the remainder of the translation unit. An exception could, of course, be made for variables declared at namespace scope, but such a variance in the rules would be surprising and potentially problematic for code refactoring. Consider replacing several namespace scope variables with static local variables.

The examples above illustrate cases where the author feels *consistent resolution* results in rejection of code that should be well-formed. Further, when the desire for *consistent resolution* is present, the author feels such constraints should be explicitly specified separately from general satisfaction of a concept. Finally, since the need to specify all of *consistent resolution*, *independent resolution*, and *matched resolution* requirements exists, the author feels more work is needed. In the mean-time, relaxing the requirement for *consistent resolution* will enable more code to be well-formed without having to resort to the current unsatisfactory workarounds.

Wording

☐ Hide deleted text

These changes are relative to [N4674](#) [\[N4674\]](#). If [P0696R0](#) [\[P0696R0\]](#) is adopted, then additional changes relative to the wording adopted in that paper will be required.

Remove the added paragraph 4 inserted in 10.1.7.4 [dcl.spec.auto]:

~~Add the following after paragraph 3 to describe when *constrained-type-specifiers* in the return type refer to template parameters:~~

~~A *constrained-type-specifier* *C1* within the declared return type of an abbreviated function template declaration does not designate a placeholder if its introduced *constraint-expression* (10.1.7.4.2) is determined to be equivalent, using the rules in 17.6.5.1 for comparing expressions, to the introduced *constraint-expression* for a *constrained-type-specifier* *C2* in the *parameter-declaration-clause* of that function declaration. Instead, *C1* is replaced by the template parameter invented for *C2* (11.3.5). [Example:~~

```
template<typename T> concept bool C = true;
```

```
template<typename... T> struct Tuple;
```

```
C const& f1(C); // has one template parameter and no deduced return type
Tuple<C...> f2(C); // has one template parameter and a deduced return type
```

In the declaration ~~¶1~~, the *constraint-expression* introduced by the *constrained-type-specifiers* in the *parameter-declaration-clause* and return type are equivalent ; they would both introduce the expression $C\langle T \rangle$, for some invented template parameter T . In ~~¶2~~, the use of C in the return type would introduce the *constraint-expression* $(C\langle T \rangle \ \&\&1 \dots)$, which is distinct from the *constraint-expression* $C\langle T \rangle$ introduced by the invented *constrained-parameter* (17.1) for the *constrained-type-specifier* in the *parameter-declaration-clause* according to the rules in 11.3.5. — end example

Renumber the subsequent paragraphs 5-16 in 10.1.7.4 [dcl.spec.auto] to reflect the removal of paragraph 4 and to align the paragraph numbers with the C++ standard.

In 10.1.7.4.1 [dcl.spec.auto.deduct] paragraph 3 (corresponding to paragraph 4 in the C++ standard), modify the last example to indicate that each *constrained-type-specifier* corresponds to a different invented template parameter:

[...]

[Example:

```
auto cf(int) -> Pair<C, C> { return expr; }
```

The return type of `cf` is deduced from the parameter `p2` in the call `f2(expr)` of the following invented function:

```
template<C T> void f2(Pair<T, T>);  
template<C T1, C T2> void f2(Pair<T1, T2>);
```

Both *constrained-type-specifiers* in the return type of `cf` correspond to ~~the same~~ **different** invented template parameters.

Change the text added as 11.3.5 [dcl.fct] paragraph 19:

Each template parameter is invented as follows.

- (19.1) — If the placeholder is designated by the *auto type-specifier*, then the invented template parameter is a type *template-parameter*.
- (19.2) — Otherwise, the placeholder is designated by a *constrained-type-specifier*, and the invented parameter is a *constrained-parameter* (17.1) whose *qualified-concept-name* is that of the *constrained-type-specifier*.
- (19.3) — If the placeholder appears in the *decl-specifier-seq* of a function parameter pack (??), or the *type-specifier-seq* of a *type-id* that is a pack expansion, the invented template parameter is a template parameter pack.

All placeholders designated by *constrained-type-specifiers* ~~whose corresponding constrained-parameters would introduce equivalent constraint-expressions (17.1), using the rules for comparing expressions in 17.6.5.1,~~ have ~~the same~~ **unique** invented template parameters

[Example:

```
template<typename T> concept bool C = true;  
template<typename, typename> class ct {};  
  
void f1(C a, C b);  
void f2(ct<C,C>);  
C f3(C c);
```

The types of `a`, `b`, `c`, and the template arguments to `ct` each correspond to unique invented template type parameters. The return type of `f3` is deduced from non-discarded return statements, if any, in

the body of the function (9.4.1). — end example] [Example:

```
namespace N {  
  template<typename T> concept bool C = true;  
}  
template<typename T> concept bool C = true;  
template<typename T, int> concept bool D = true;  
template<typename, int = 0> concept bool E = true;  
void abbr(C, D<0>);
```

The *constrained-type-specifiers* `C` and `D<0>` correspond to distinct invented template parameters in the declaration of `abbr`.

```
void f0(C a, C b);
```

The types of `a` and `b` are the same invented template type parameter.

```
void f1(C& a, C* b);
```

The type of `a` is a reference to an invented template type parameter `T`, and the type of `b` is a pointer to `T`.

```
void f2(N::C a, C b);  
void f3(D<0> a, D<1> b);
```

In both functions, the parameters `a` and `b` have different invented template type parameters.

```
void f4(E a, E<> b, E<0> c);
```

The types of `a`, `b`, and `c` are the same because the *constrained-type-specifiers* `E`, `E<>`, and `E<0>` all associate the *constraint-expression* `E<T, 0>`, where `T` is an invented template type parameter.

```
void f5(C head, C... tail);
```

The types of `head` and `tail` are different. Their respective introduced *constraint-expressions* are `C<T>` and `(C<U> && ...)`, where `T` is the template parameter invented for `head` and `U` is the template parameter invented for `tail` (17.1).

Acknowledgements

The author would like to thank Andrew Sutton for his continued dedication to progressing the adoption of Concepts into the C++ working paper.

Thanks to Botond Ballo for initial draft review and feedback.

References

- [N3878] "Extensions to the Concept Introduction Syntax in Concepts Lite"
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3878.pdf>

- [N4674] "Working Draft, C++ Extensions for concepts"
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4674.pdf>
- [P0464R2] "Revisiting the meaning of `foo(ConceptName, ConceptName)`"
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0464r2.html>
- [P0691R0] "Integrating Concepts: “Open” items for consideration"
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0691r0.pdf>
- [P0694R0] "Function declarations using concepts"
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0694r0.pdf>
- [P0696R0] "Remove abbreviated functions and template-introduction syntax from the Concepts TS"
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0696r0.html>
- [GccBug81270] "[concepts] ill-formed code with a constrained variable declaration with multiple declarators with different deduced types not rejected"
https://gcc.gnu.org/bugzilla/show_bug.cgi?id=81270